

Music Scheduler Development Environment - READY!

The complete development environment has been successfully set up and is now running!

Access Information

Application URLs

- 🎵 **Music Scheduler Frontend:** `http://localhost:3000`
- 🔧 **Backend API:** `http://localhost:5000`
- ❤️ **Health Check:** `http://localhost:5000/api/health`

Server Status

- ✅ **Flask Backend:** Running on port 5000
- ✅ **React Frontend:** Running on port 3000
- ✅ **MySQL Database:** Connected and operational
- ✅ **API Health:** "API is running and database is connected"

Test Accounts

Admin Account

- **Username:** `admin`
- **Password:** `admin123`
- **Role:** Admin (full access)

Instructor Accounts

- **Username:** `john_instructor`
- **Password:** `instructor123`
- **Role:** Instructor
- **Instruments:** Piano, Guitar
- **Availability:** Monday, Wednesday, Friday (9 AM - 5 PM)
- **Username:** `jane_instructor`
- **Password:** `instructor123`
- **Role:** Instructor
- **Instruments:** Violin, Piano
- **Availability:** Tuesday, Thursday, Saturday (10 AM - 6 PM / 9 AM - 3 PM Sat)

Sample Data

Students (5 total)

1. **Emma Watson** - `emma@example.com`, Piano
2. **Liam Johnson** - `liam@example.com`, Guitar

3. **Olivia Brown** - olivia@example.com, Violin
4. **Noah Davis** - noah@example.com, Drums
5. **Ava Wilson** - ava@example.com, Piano

Lessons (3 scheduled for next week)

1. Emma Watson - Piano lesson with John (Monday 10:00 AM, 60 min)
2. Liam Johnson - Guitar lesson with John (Wednesday 2:00 PM, 45 min)
3. Olivia Brown - Violin lesson with Jane (Tuesday 11:00 AM, 60 min)

Development Commands

Stop Servers

```
# Find and kill development servers
pkill -f flask
pkill -f vite
pkill -f node
```

Restart Servers

```
cd ~/music-scheduler
./run_dev.sh
```

Check Logs

```
# Flask backend logs
tail -f flask.log

# Vite frontend logs
tail -f vite.log

# Setup logs
cat setup.log
```

Database Access

```
# Connect to database
mysql -u music_user -p'music_pass' music_scheduler

# Quick queries
mysql -u music_user -p'music_pass' music_scheduler -e "SELECT COUNT(*) FROM users;"
mysql -u music_user -p'music_pass' music_scheduler -e "SELECT COUNT(*) FROM students;"
mysql -u music_user -p'music_pass' music_scheduler -e "SELECT COUNT(*) FROM lessons;"
```

API Testing

Health Check

```
curl http://localhost:5000/api/health
```

Login Test

```
curl -X POST http://localhost:5000/api/login \
  -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "admin123"}'
```

Get Students

```
curl -b cookies.txt http://localhost:5000/api/students
```

Project Structure

music-scheduler/	
 app.py	# Flask backend (main application)
 vite.config.js	# Vite configuration with API proxy
 package.json	# Node.js dependencies
 requirements.txt	# Python dependencies
 .env	# Environment configuration
 setup_dev.sh	# Setup script (completed)
 run_dev.sh	# Development server runner
 SETUP.md	# Comprehensive setup guide
 DEVELOPMENT_READY.md	# This file
 venv/	# Python virtual environment
 sessions/	# Flask session storage
 node_modules/	# Node.js dependencies (178 packages)
 src/	# React source code
 index.jsx	# Main React component (Admin Dashboard)
 index.css	# Styling
 main.jsx	# React entry point
 logs/	# Application logs

Development Features

Backend (Flask)

-  **Authentication:** Session-based with bcrypt password hashing
-  **Database:** MySQL with complete schema and foreign keys
-  **API:** RESTful endpoints for users, students, lessons, availability
-  **CORS:** Configured for React frontend communication
-  **Environment:** Uses .env for configuration
-  **Health Check:** /api/health endpoint for monitoring

Frontend (React + Vite)

-  **React 18:** Modern React with hooks
-  **Calendar:** React-Calendar component for lesson scheduling
-  **Styling:** Tailwind CSS for modern UI
-  **Hot Reload:** Automatic refresh on code changes
-  **API Proxy:** Vite proxies /api calls to Flask backend
-  **Admin Dashboard:** Complete admin interface for managing data

Database (MySQL)

-  **Users:** Admin and instructor accounts with roles
-  **Students:** Contact info and instrument preferences
-  **Lessons:** Scheduled lessons with all details
-  **Availability:** Instructor weekly availability patterns
-  **Instruments:** Instruments taught by each instructor
-  **Time Off:** Instructor time-off periods



Security Features

-  **Password Hashing:** Bcrypt for secure password storage
-  **Session Management:** Secure filesystem-based sessions
-  **CORS:** Restricted to localhost:3000 for development
-  **SQL Security:** Parameterized queries prevent injection
-  **Environment Variables:** Database credentials in .env file



Next Steps

1.  **Open the application:** `http://localhost:3000`
2.  **Login with admin account:** `admin / admin123`
3.  **Explore the Admin Dashboard:**
 - View and manage users
 - Add and edit students
 - Schedule lessons
 - View calendar
 - Manage instructor availability
4.  **Test API endpoints** using curl or Postman
5.  **Start developing** new features!



Troubleshooting

If servers stop working:

1. Check if MySQL is running: `sudo service mysql status`
2. Restart development servers: `./run_dev.sh`
3. Check logs: `tail -f flask.log vite.log`

If database issues:

1. Test connection: `mysql -u music_user -p'music_pass' music_scheduler`
2. Check tables: `SHOW TABLES;`
3. Verify data: `SELECT COUNT(*) FROM users;`

🎵 Your Music Scheduler Development Environment is Ready! 🎵

Frontend: `http://localhost:3000`

Backend: `http://localhost:5000`

Login: admin / admin123

Happy coding! 🚀